浙江大学
ZHEJIANG UNIVERSITY



“智学引擎”智能学习助手系统
系统设计报告

组　长　陈子涵

组　员　邓佳骏 方李庭 郭云海 魏文俐

日　期　2025.11.29

版　本　version 1.0

修改历史

修订日期	版本号	修改人	修改内容	审核人
2025-11-29	V1.0	G09	报告整合完成	

Contents

1 引言 **4**

1.1 编写目的 4

1.2 软件项目背景 4

2 需求规定 **4**

2.1 用户需求 4

2.2 其它需求 5

3 总体设计 **5**

3.1 功能设计 5

3.2 用户类型及用户特征 6

3.3 运行环境 6

3.3.1 软件运行环境 6

3.3.2 硬件运行环境 7

3.4 基本概念和处理流程 7

3.5 结构 8

3.5.1 用户需求分析图 8

3.5.2 系统模块架构图 9

3.5.3 数据流图 10

3.5.4 ER 图 11

3.5.5 关键 IPO 图 12

4 接口设计 **13**

4.1 用户接口 13

4.2 外部接口 14

4.3 内部接口 14

4.3.1 核心问答接口 14

4.3.2 文档管理与 AI 工具接口 14

4.3.3 扩展服务接口 15

5 总体数据设计 **15**

5.1 数据存储 15

5.2 数据安全 15

5.3 逻辑结构设计要点 16
5.4 物理结构设计要点 16

6 模块设计计划 16
6.1 项目架构设计 16
6.2 项目任务分解 17
6.3 前端 18
6.4 后端 19

1 引言

1.1 编写目的

本报告旨在全面阐述“智学引擎”智能学习助手系统的总体技术架构与详细设计方案，将项目计划中的业务需求转化为可落地的工程规范。通过明确模块划分、数据结构设计与接口定义（FastAPI/Next.js），统一开发团队对系统实现的技术认知，为后续的编码实现、系统集成与测试验收提供标准化的指导依据，确保项目能够按计划交付高质量、可溯源且易于维护的软件成果。

1.2 软件项目背景

在当前的高校教育与个人自学场景中，学习资料往往以 PDF、Markdown 等多种格式分散存储，学习者需要在课件、笔记与复习大纲之间频繁切换，导致知识获取碎片化，难以建立系统的知识体系。传统的关键词检索工具无法理解资料的语义关联，而通用的生成式 AI 模型在缺乏私有领域知识时容易产生“幻觉”，且回答缺乏可追溯的来源依据，无法满足严谨的学习需求。

针对上述问题，本项目基于检索增强生成（RAG）技术，利用 LlamaIndex 框架结合 DeepSeek 大语言模型与 BAAI 重排序模型，构建“智学引擎”全栈 Web 应用。系统旨在为用户建立一个私有化、可交互的本地知识库，通过向量索引与语义重排序技术实现高精准度、可溯源的智能问答。此外，系统深度集成了文档智能摘要、自动测验生成、划词精准问答及 Semantic Scholar 学术文献检索功能，构建覆盖“知识输入—深度理解—巩固内化—边界拓展”的全链路闭环，推动学习模式从被动接收向主动探索转变。

2 需求规定

2.1 用户需求

- **在校学生（深度复习与备考）：**需支持对 PDF/Markdown 文件的上传与解析，通过智能问答获取附带来源引用的准确解答。需利用文档阅读器的“划词问答”功能，针对特定段落进行上下文精准解析，并借助自动生成的摘要与测验题巩固知识点。
- **自学者（知识管理与内化）：**需构建私有的本地化知识库，将非结构化文档转化为向量索引。支持通过自然语言对个人资料库进行深度检索，快速提取核心概念，实现从被动阅读到主动知识管理的转变。
- **科研人员（学术边界拓展）：**需集成 Semantic Scholar 外部 API，实现课内知识与前沿学术论文的无缝衔接。支持基于关键词的实时文献检索，获取论文标题、摘

要及原文链接，辅助文献综述与研究。

2.2 其它需求

- **性能需求：**基于 FastAPI 的异步处理能力，支持高并发下的低延迟响应。常规问答响应时间 ≤3 秒，长文档（8000 字符以上）摘要生成时间 ≤5 秒，静态资源加载时间 ≤2 秒。
- **并发需求：**系统需在 50 人并发场景下保持稳定。利用 FastAPI 的异步特性（Async/Await）处理 I/O 密集型任务（如 LLM 调用、数据库查询），确保在多用户同时执行文档上传与问答时，服务不发生阻塞或崩溃。
- **安全需求：**
  - **接口安全：**配置 CORS（跨域资源共享）白名单，仅允许受信任的前端域名访问后端 API。
  - **密钥管理：**所有敏感凭证（如 `DEEPSEEK_API_KEY`）必须通过环境变量（.env）加载，严禁硬编码在代码中。
  - **数据隐私：**用户上传的原始文档与向量索引仅存储于服务器本地（`./data` 与 `./storage`），不经由第三方云存储中转，保障私有数据安全。
- **质量需求：**系统核心功能（问答、检索、生成）的可用性需达到 99.9%。前端界面需适配 Chromium 内核主流浏览器（Chrome, Edge），并具备完善的异常处理机制，在 LLM 服务超时或文件解析失败时提供友好的错误提示。
- **可扩展性需求：**采用前后端分离架构（Next.js + FastAPI）。后端需预留标准 RESTful 接口，支持未来接入用户认证系统（JWT）、扩展新的文件解析器（如 Word/PPT）以及更换底层 LLM 模型（如 GPT-4 或本地模型）而不影响上层业务逻辑。

3 总体设计

3.1 功能设计

- **智能问答中心（RAG Core）：**系统基于 LlamaIndex 框架构建 RAG 管道。用户提问时，系统首先通过 `BAAI/bge-base-zh-v1.5` 模型将问题向量化，并在向量索引中召回 Top-5 相关文档片段；随后利用 `BAAI/bge-reranker-base` 模型对召回结果进行语义重排序（Rerank），最终将精准的上下文输送给 `DeepSeek-chat` 大语言模型，生成携带来源标注的回答。

- **学习资料中心（Knowledge Management）：**支持 PDF/Markdown 等非结构化文档的上传与管理。后端采用 `SimpleDirectoryReader` 实时解析文件，并增量更新向量索引。针对单个文档，集成 Prompt Engineering 能力，提供“一键智能摘要”和“自动生成测验题”功能，辅助知识内化。
- **交互式文档阅读器（Contextual Learning）：**内置 PDF 渲染引擎，支持“划词问答”特性。用户在阅读过程中选中任意文本段落，即可触发上下文对话（Contextual Chat），后端仅基于选中文本调用 LLM 进行解释或扩展，实现颗粒度更细的知识交互。
- **学术文献检索（Academic Extension）：**打破本地知识库边界，通过后端代理（Proxy）集成 Semantic Scholar API。支持基于关键词的实时学术搜索，返回论文标题、作者、摘要及原文链接，构建“课内资料 + 前沿学术”的双源知识体系。

3.2 用户类型及用户特征

用户类	特征与说明
学习者	系统的核心用户。拥有知识库的读写权限，能够上传个人学习资料构建私有索引，通过问答与阅读器进行深度学习。关注问答的准确性与来源的可追溯性。
系统管理员	负责后端服务的部署与维护。拥有服务器底层文件系统的访问权限，负责管理 `.env` 配置文件中的 API 密钥（DeepSeek Key），以及运行 `build_index.py` 脚本进行索引的初始化构建与重建。

3.3 运行环境

3.3.1 软件运行环境

项目	名称与版本
操作系统	Windows 为主，也可在 Linux 或 macOS 运行，推荐容器化部署以简化依赖管理。
后端运行时	Python 3.10, FastAPI 0.121.1, Uvicorn 0.38.0, 依赖 LlamaIndex 0.14.7, httpx 0.28.1。
前端运行时	Node.js 18 及以上, Next.js 16.0.1, React 19.2.0, Tailwind CSS 4。
模型与向量服务	DeepSeek Chat 模型通过云端 API 调用，向量与重排序模型使用 BAAI/bge-base-zh-v1.5 与 BAAI/bge-reranker-base 本地推理，需安装 PyTorch。
数据与存储	文档存放于后端 `./data` 目录，向量索引持久化于 `./storage`，无额外数据库依赖。
浏览器兼容性	已在 Chromium 内核浏览器验证（Chrome、Edge），支持现代 ES2020 特性。

3.3.2 硬件运行环境

项目	推荐配置
计算节点	4 核 CPU, 16GB 内存；若需要更快的向量插入与重排序，可接入 1 块支持 CUDA 的中端 GPU。
存储	系统盘与数据盘合计 20GB 以上可用空间，保证 `./data` 与 `./storage` 有充足余量。
网络	出站 HTTPS 连通性以访问 DeepSeek 与 Semantic Scholar API，内网带宽 100 Mbps 以上以支撑文件上传与 PDF 传输。
部署形态	单节点即可支撑当前功能；未来可将前后端分容器部署，结合反向代理实现水平扩展与证书管理。

3.4 基本概念和处理流程

关键概念：

- **知识库文档：**指用户上传并通过向量化处理的 PDF 或 Markdown 文件。
- **向量索引：**是以段落为粒度的向量存储与检索结构。
- **重排序器：**用于在召回结果上基于语义相似度重新排序。
- **语境问答：**在已有上下文内直接生成答案，不访问全局索引。
- **学术检索：**通过外部论文库扩充资料边界。

处理流程：

(1). **文档入库：**用户在学习资料中心上传文件，后端保存到 `./data`，使用 SimpleDirectoryReader 切分并插入索引，持久化到 `./storage`，返回上传结果。

(2). **全局问答：**用户在智能问答中心提出问题，前端调用 `/api/chat`，后端用向量索引召回前五个片段、经重排序器筛选，DeepSeek 生成回答与引用片段，前端按消息流式展示并列出来源。

(3). **文档摘要与测验：**在资料中心选择单个文件，前端调用文档摘要接口或测验接口，后端截取文档前 8000 字符，生成摘要或三道选择题并返回，前端以对话框呈现。

(4). **划词问答：**用户在阅读器中选中文本，填写问题后调用 `/api/contextual_chat`，后端仅依据选中文本生成答案，确保解释与原文一致。

(5). **学术文献检索：**前端将关键词传递至检索接口，后端代理 Semantic Scholar，解析论文标题、作者、摘要与链接返回前端。

3.5 结构

3.5.1 用户需求分析图

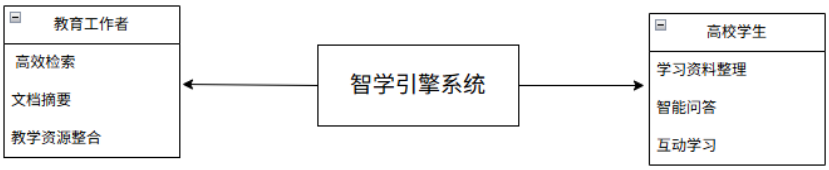


Figure 1: 用户需求分析图

如图所示，“智学引擎系统”作为核心枢纽，旨在同时满足教育侧与学习侧的双重需求，连接了**教育工作者**与**高校学生**两大关键用户群体：

(1). **教育工作者（左侧）：**

- **高效检索：**针对海量的教学文献与科研资料，教育工作者需要系统提供快速、精准的语义检索能力，以节省查阅时间。
- **文档摘要：**利用 AI 自动生成摘要功能，辅助教师快速提炼文档核心观点，提升备课与文献筛选效率。
- **教学资源整合：**支持将分散的课件、大纲与参考书目上传至系统构建统一知识库，实现教学资源的结构化管理与复用。

(2). **高校学生（右侧）：**

- **学习资料整理：**学生可构建私有的学习资料库，将不同课程的 PDF 或 Markdown 笔记集中管理，解决资料碎片化问题。
- **智能问答：**面对复习中的疑难点，学生通过 RAG 技术获取基于课程内容的精准解答与来源追溯，实现个性化辅导。
- **互动学习：**通过“划词问答”与“自动测验”等功能，系统支持学生与资料进行深度交互，推动从被动阅读向主动探索的转变。

3.5.2 系统模块架构图

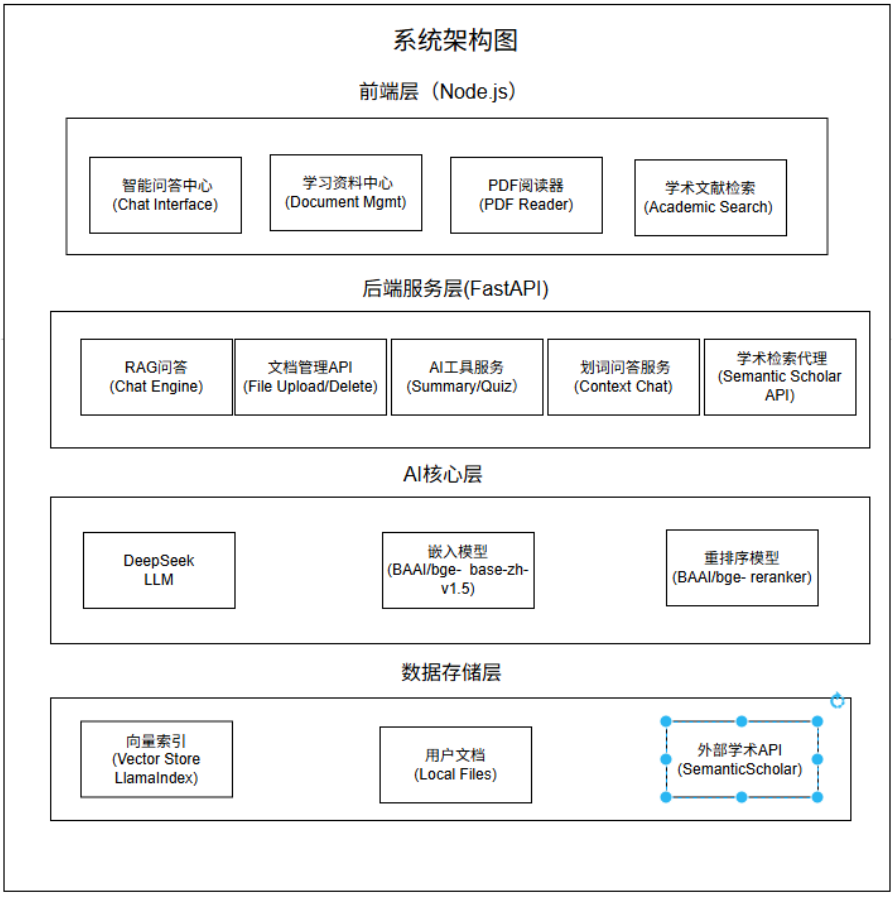


Figure 2: 系统模块架构图

如图所示，我们采用经典的分层架构设计，自上而下划分为四个逻辑层，各层职责分明，确保了系统的高内聚低耦合：

- **前端层 (Node.js)：**基于 Node.js 运行时环境（Next.js 框架）构建。主要包含四大交互模块：“智能问答中心”提供对话交互，“学习资料中心”负责文档管理，“PDF 阅读器”实现文档渲染与划词交互，“学术文献检索”提供外部学术资源入口。

- **后端服务层 (FastAPI)：**作为系统的业务逻辑中枢，采用 Python FastAPI 框架。它向下调度 AI 能力，向上提供 RESTful 接口。核心服务包括处理 RAG 流程的“Chat Engine”、管理文件上传删除的“文档管理 API”、生成摘要与测验的“AI 工具服务”、支持局部上下文的“划词问答服务”以及代理外部请求的“学术检索代理”。

- **AI 核心层：**封装了系统智能化的核心组件。其中 **DeepSeek LLM** 负责自然语言生成与逻辑推理；**嵌入模型 (BAAI/bge-base-zh-v1.5)** 负责将文本转化为高维向量；**重排序模型 (BAAI/bge-reranker)** 负责对检索结果进行精细化排序，以提升问答准确性。

- **数据存储层：**负责数据的持久化存储与外部数据源接入。系统采用本地化存储策略，“用户文档”直接存储于文件系统（./data），“向量索引”通过 LlamaIndex 持久化存储（./storage）。同时，该层也包含对外部“SemanticScholar API”的数据接入能力，作为本地知识库的补充。

3.5.3 数据流图

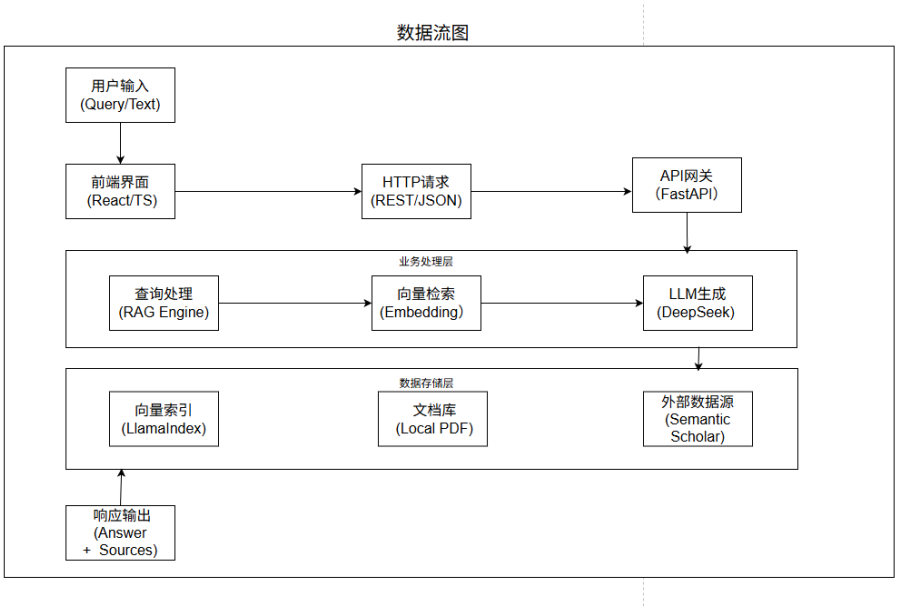


Figure 3: 数据流图

如图所示，系统的数据流向遵循清晰的“请求—处理—响应”闭环，贯穿前端、后端、业务逻辑与数据存储四层结构：

- **请求发起与传输：**用户在前端界面（React/TS）输入查询（Query）或文本（Text），系统将其封装为 JSON 格式，通过 HTTP 协议发送至后端 API 网关（FastAPI）。

- **业务逻辑处理：** API 网关接收请求后，将其分发至业务处理层。查询处理模块（RAG Engine）首先调用向量检索服务（Embedding），将自然语言转化为向量表示，确立语义搜索上下文；随后，系统将检索到的相关信息输送至大语言模型（DeepSeek）进行生成式处理。
- **数据存取交互：** 业务层在处理过程中深度交互数据存储层。向量检索依赖 LlamaIndex 管理的向量索引，并回溯至文档库（Local PDF）获取原始内容；对于学术检索请求，系统则连接外部数据源（Semantic Scholar）获取实时论文数据。
- **响应输出：** 经过 LLM 生成与来源匹配后，系统构建包含最终回答（Answer）与来源标注（Sources）的响应对象，并沿原路返回至前端进行渲染展示。

3.5.4 ER 图

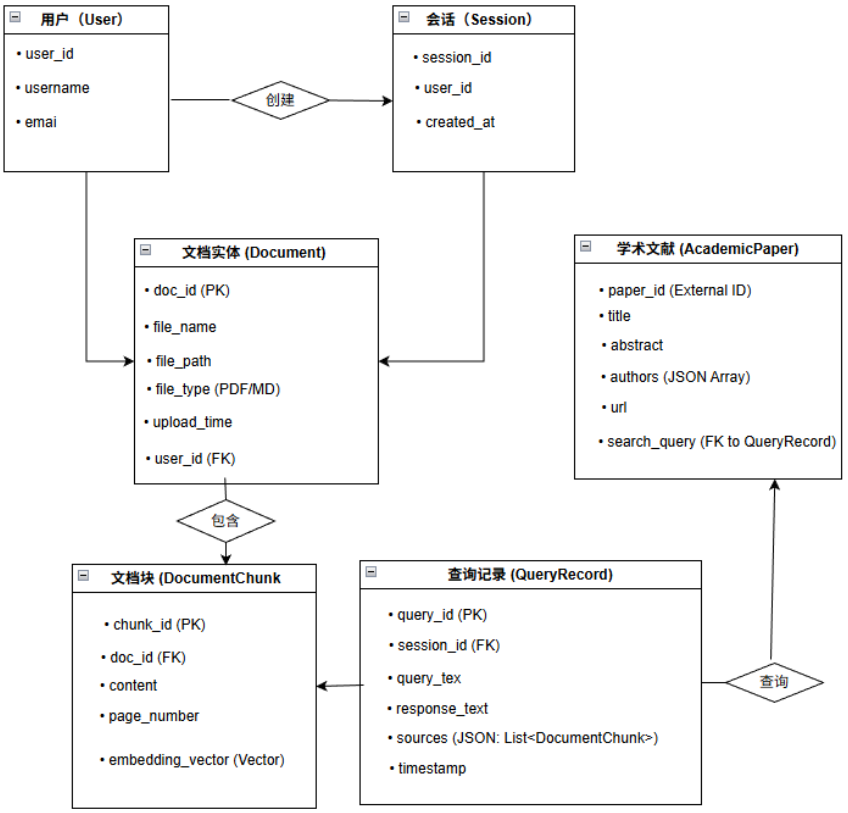


Figure 4: ER 图

如图所示，系统数据库设计围绕“用户—文档—问答”三条核心主线展开，各实体间的逻辑关系如下：

- **用户与会话管理：** **用户 (User)** 是系统的核心主体，负责创建和管理个人的 **会话 (Session)**。一个用户可以发起多个会话，用于隔离不同主题的学习交互历史。

- **文档存储体系：**用户上传的学习资料被抽象为 **文档实体 (Document)**，存储文件名、类型与路径等元数据。为了支持 RAG 向量检索，每个文档通过“包含”关系被拆分为多个更细粒度的 **文档块 (DocumentChunk)**。文档块不仅存储具体的文本内容与页码，还包含用于语义搜索的关键字段 `embedding_vector`。
- **问答交互记录： 查询记录 (QueryRecord)** 依附于会话存在，详细记录了用户的提问 (`query_text`) 与系统的回复 (`response_text`)。关键在于，它通过 JSON 格式的 `sources` 字段关联回具体的 **文档块**，实现了问答结果的可溯源性。
- **学术知识扩展： 学术文献 (AcademicPaper)** 代表通过外部 API 获取的论文资源。它通过“查询”关系与 **查询记录**动态关联，存储论文的标题、摘要及作者信息，作为本地知识库的有效补充。

3.5.5 关键 IPO 图

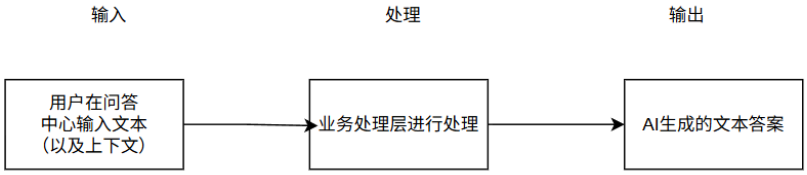


Figure 5: IPO 图 1 全局问答

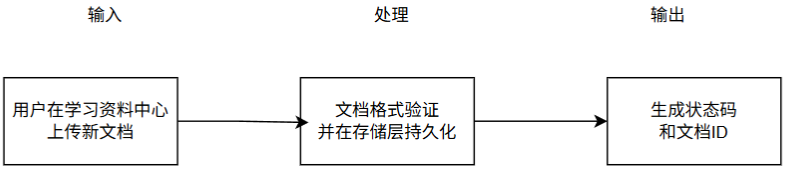


Figure 6: IPO 图 2 文档上传与索引

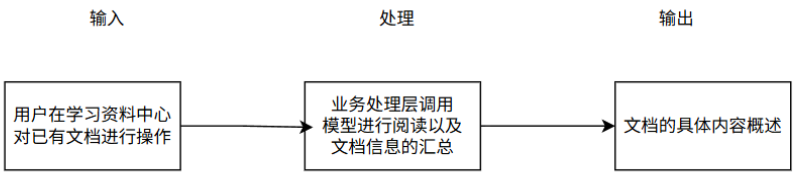


Figure 7: IPO 图 3 摘要测验与划词

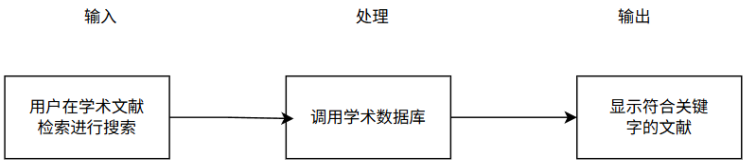


Figure 8: IPO 图 4 学术文献检索

场景	输入	输出
全局问答	用户问题，向量索引内容	经过重排序的引用片段与 DeepSeek 生成的回答，前端对话消息
文档上传与索引	PDF 或 Markdown 文件	存储于 `./data` 的原始文件、持久化的向量索引、上传确认信息
文档摘要与测验	指定文件内容前 8000 字符	Markdown 摘要或三道单项选择题列表，用于对话框展示
划词问答	被选中的文本片段、用户问题	基于局部上下文的精准回答，附带引用片段
学术文献检索	关键词	论文标题、作者、摘要与外部链接列表

4 接口设计

4.1 用户接口

本系统前端采用 **Next.js** 框架构建，通过 RESTful API 与后端交互，提供响应式、流式渲染的用户体验。主要包含以下四大功能模块：

- **智能问答中心 (Smart QA Hub)：**系统主入口，提供类似 ChatGPT 的对话界面。支持流式输出回答，并在每条回答下方以卡片形式展示“参考来源”（文件名、页码及文本片段），确保信息可溯源。
- **学习资料中心 (Resource Dashboard)：**知识库管理看板。以列表形式展示 `./data` 目录下的文件，提供“文件上传”、“删除”及“在线预览”入口。界面集成 AI 工具栏，允许用户对单篇文档触发“生成摘要”或“生成测验”指令。
- **文档阅读器 (PDF Reader)：**内嵌 PDF 渲染组件。核心交互为“划词问答”：用户鼠标选中一段文本后，前端自动捕获该段落作为 `context`，弹出悬浮对话框供用户针对该段落提问。
- **学术文献检索 (Academic Search)：**独立的搜索界面。用户输入关键词后，系统异步请求外部数据库，并以列表形式展示论文标题、作者、摘要及 URL 链接。

4.2 外部接口

系统后端依赖以下外部服务来增强核心能力：

- **DeepSeek API (LLM Service)：**
  - **用途：**系统的智能核心，负责 RAG 问答生成、文档摘要提炼及测验题构造。
  - **调用方式：**后端通过 `llama-index.llms.deepseek` 库封装调用，API Key 通过环境变量 `DEEPSEEK_API_KEY` 管理。
- **Semantic Scholar API (Academic Graph)：**
  - **用途：**扩展知识边界，提供实时学术论文检索数据。
  - **调用方式：**后端作为代理（Proxy），向 `api.semanticscholar.org` 发起 HTTP GET 请求，并将返回的 JSON 数据清洗为系统标准格式。

4.3 内部接口

系统采用前后端分离架构，后端基于 **FastAPI** 提供 RESTful 接口。以下为核心 API 定义及数据模型规范：

4.3.1 核心问答接口

POST /api/chat

- **描述：**全局 RAG 问答接口，基于向量索引检索知识库。
- **请求体：** `ChatRequest { query: str }`。
- **响应体：** `ChatResponse`，包含生成的 `answer` 和 `sources`（含 `file_name`, `page_label`, `text_snippet`）。

POST /api/contextual_chat

- **描述：**划词问答接口，仅基于前端传入的上下文片段回答。
- **请求体：** `ContextualChatRequest { context: str, query: str }`。
- **响应体：** `ContextualChatResponse { answer: str }`。

4.3.2 文档管理与 AI 工具接口

POST /api/upload

- **描述：**接收 `multipart/form-data` 文件，保存至 `./data` 并触发增量向量索引构建。

DELETE /api/documents/{file_name}

- **描述：**删除物理文件并从向量存储（Vector Store）中移除对应节点。

GET /api/documents/{file_name}/summary

- **描述：**截取文档前 8000 字符，调用 LLM 生成文本摘要。

GET /api/documents/{file_name}/quiz

- **描述：**基于文档内容生成 JSON 格式的单项选择题列表。
- **响应体：** `List[QuizQuestion]`，包含 `question`, `options`, `answer` 。

4.3.3 扩展服务接口

GET /api/academic_search

- **描述：**接收 `query` 参数，返回结构化的论文列表。
- **响应体：** `List[AcademicPaper]`，包含 `title`, `abstract`, `authors`, `url` 。

5 总体数据设计

5.1 数据存储

本系统摒弃了传统的关系型数据库依赖，采用了基于 LlamaIndex 的轻量级本地文件存储策略，确保了部署的便捷性与数据的私有性：

- **非结构化文档库 (Raw Data Store)：**用户上传的 PDF、Markdown 等原始学习资料直接存储于后端服务器的 `./data` 目录。该目录作为知识库的“源头”，支持 `SimpleDirectoryReader` 进行增量读取。
- **向量索引库 (Vector Index Store)：**经切分（Chunking）与嵌入（Embedding）后的高维向量数据及关联元数据（Metadata），由 LlamaIndex 的 `StorageContext` 统一管理，并以 JSON/二进制格式持久化存储于 `./storage` 目录。这构成了 RAG 检索的核心数据库，无需外部向量数据库（如 Pinecone）支持。

5.2 数据安全

- **接口访问控制 (CORS)：**后端配置了严格的跨域资源共享策略（`CORSMiddleware`），白名单仅放行 `http://localhost:3000` 等受信任的前端域，有效防止 CSRF 攻击。

- **敏感凭证隔离：**系统的核心密钥（如 DEEPSEEK_API_KEY）强制通过 .env 环境变量文件加载，代码库中不包含任何硬编码密钥，确保开源或协作时的信息安全。
- **数据隐私保护：**所有的知识库构建与向量计算均在服务器本地完成（Local Storage），原始文档与向量索引不上传至第三方云数据库，最大限度保障用户私有数据的安全。

5.3 逻辑结构设计要点

Smart-Learning-Engine 的数据流转遵循“文档 → 节点 → 索引 → 查询”的标准链路：

- **文档对象 (Documents)：**对应 ./data 中的物理文件，包含全量文本内容。
- **文本节点 (Nodes)：**文档经切分后的最小检索单元。每个节点封装了文本片段（Chunk）以及 file_name, page_label 等关键元数据，用于后续的来源溯源。
- **语义向量 (Embeddings)：**由 BAAI/bge-base-zh-v1.5 模型将文本节点编码为 768 维向量，用于计算余弦相似度。
- **查询引擎 (Query Engine)：**由向量索引（VectorStoreIndex）与重排序器（BAAI/bge-reranker-ba Top-N=3）组合而成，负责在逻辑上连接用户 Query 与底层数据。

5.4 物理结构设计要点

系统文件物理布局严格遵循功能分离原则：

(1). **数据区 (./data)：**存放用户上传的原始文件，支持直接的文件系统操作（如 OS 级删除、备份）。

(2). **索引区 (./storage)：**存放 LlamaIndex 生成的持久化文件（docstore.json, index_store.json, vector_store.json 等），视作系统的“本地数据库实例”。

(3). **应用区：**前端静态资源由 Next.js 构建生成，后端业务逻辑由 FastAPI 承载，两者通过 HTTP 协议交换 JSON 数据，不共享物理存储空间。

6 模块设计计划

6.1 项目架构设计

系统遵循高内聚、低耦合原则，采用现代化的前后端分离架构，自上而下划分为四层：

- **前端交互层 (Presentation Layer)：**基于 **Next.js 14** (App Router) 框架构建，使用 **TypeScript** 保障类型安全。UI 组件库采用 **shadcn/ui** (Radix UI) 结合 **Tailwind CSS** 实现原子化样式管理，确保界面美观与响应式适配。
- **后端服务层 (Service Layer)：**以 **FastAPI** 为核心框架，利用其 `async/await` 特性处理高并发请求。通过 **Pydantic** 模型定义数据契约，使用 **Uvicorn** 作为 ASGI 服务器进行部署。
- **AI 核心层 (Intelligence Layer)：**基于 **LlamaIndex** 编排 RAG 流程。集成 **DeepSeek-chat** 作为推理核心，**BAAI/bge-base-zh-v1.5** 负责向量嵌入，**BAAI/bge-reranker-base** 负责检索结果的重排序优化。
- **数据存储层 (Persistence Layer)：**采用“本地文件 + 向量索引”的轻量化存储方案。原始文档存至 `./data`，向量索引通过 LlamaIndex 持久化至 `./storage`，无需外部数据库依赖，保障数据隐私。

6.2 项目任务分解

模块名称	核心任务与交付物	依赖资源	完成周期
基础架构搭建	1. 初始化 Next.js 前端仓库与 FastAPI 后端服务。 2. 配置 CORS 跨域策略与环境变量管理 (.env)。 3. 定义前后端通信的 RESTful 接口规范。	Git, FastAPI, Node.js	1 周
智能问答模块	1. 实现 `build_index.py` 脚本，完成向量库构建。 2. 开发 RAG 检索链路（Embedding → Rerank → Generate）。 3. 交付可运行的 MVP 问答演示版。	LlamaIndex, DeepSeek API	1 周
学习资料中心	1. 开发文件上传 (`/api/upload`) 与列表展示接口。 2. 实现后端 `SimpleDirectoryReader` 增量索引逻辑。 3. 搭建前端文档管理 Dashboard。	shadcn/ui, multipart/form-data	1 周

模块名称	核心任务与交付物	依赖资源	完成周期
AI 工具集模块	1. 设计 Prompt Template 实现文档智能摘要。 2. 开发自动测验生成接口，集成 Regex 解析 JSON 输出。 3. 前端实现摘要与测验结果的渲染组件。	Prompt Engineering, Regex	1 周
交互体验优化	1. 集成 PDF 阅读器，实现划词上下文捕获逻辑。 2. 开发 `/api/contextual_chat` 接口支持局部问答。 3. 优化流式响应 UI 与错误处理反馈。	React-PDF, Window Selection API	1 周
学术拓展模块	1. 封装 `httpx` 异步客户端调用 Semantic Scholar API。 2. 实现学术论文数据的清洗与前端列表展示。 3. 全链路联调测试与 Bug 修复。	httpx, Semantic Scholar API	1 周
测试与部署	1. 执行功能与性能测试，确保高并发稳定性。 2. 编写用户手册与部署文档。 3. 部署至云平台 (Vercel/Railway) 并上线。	Pytest, Vercel	4 天

6.3 前端

技术栈： Next.js 14, TypeScript, Tailwind CSS, shadcn/ui, React-PDF。
核心模块实现：

(1). **智能问答中心：**维护聊天状态 (`messages` 数组)，处理流式响应数据。实现“来源卡片”组件，点击可溯源至文档具体页码。

(2). **学习资料中心：**封装 `FileUpload` 组件，支持拖拽上传与上传进度条。实现文档列表的骨架屏 (Skeleton) 加载状态，提升用户体验。

(3). **文档阅读器交互：**监听 `onMouseUp` 事件，通过 `window.getSelection()` 获取用户高亮文本。结合 `Popover` 组件弹出“AI 助手”浮层，将选中文本作为 Context 发送至后端。

6.4 后端

技术栈： FastAPI, Python 3.10, LlamaIndex, httpx, Uvicorn。
核心模块实现：

(1). **文档处理流水线：** `/api/upload` 接收文件后，流式写入 `./data`，随即调用 `index.insert_nodes()` 实时更新向量库，并触发 `storage_context.persist()` 持久化索引，确保数据不丢失。

(2). **RAG 问答引擎：** `/api/chat` 接口封装了检索与生成逻辑。引入 `FlagEmbeddingReranker` 对初步召回的 Top-5 节点进行二次精排，显著提升回答的相关性。

(3). **结构化生成工具：** `/api/documents/{id}/quiz` 接口通过 Prompt 强制 LLM 输出 JSON 格式。后端增加正则匹配 (`re.search`) 逻辑，从模型回复中提取纯净 JSON 字符串，防止格式错误导致解析失败。

(4). **异步学术代理：** `/api/academic_search` 接口使用 `httpx.AsyncClient` 发起非阻塞 HTTP 请求，高效转发 Semantic Scholar 的查询结果，避免阻塞主线程。